

使用 MediaPipe 建立自訂物件偵測網頁應用程式

來源：<https://codelabs.developers.google.com/mp-object-detection-web?hl=zh-tw>

步驟數：7

瞭解如何使用 MediaPipe 建立自訂物件偵測網頁應用程式。

目錄

1. [事前準備](#)
2. [做好準備](#)
3. [匯入 MediaPipe tasks-vision 套件，並新增必要變數](#)
4. [初始化物件偵測器](#)
5. [對圖片執行預測](#)
6. [對網路攝影機即時影像執行預測](#)
7. [恭喜](#)

步驟 1 · 約 2 分鐘

1. 事前準備

[MediaPipe Solutions](#) 可讓您在應用程式中套用機器學習 (ML) 解決方案。這個框架可讓您設定預先建構的處理管道，為使用者提供即時、吸引人且實用的輸出內容。您甚至可以使用 [Model Maker](#) 自訂這些解決方案，更新預設模型。

物件偵測是 MediaPipe Solutions 提供的其中一項 ML 視覺工作。[MediaPipe Tasks](#) 適用於 Android、Python 和網頁。

在本程式碼研究室中，您將在網頁應用程式中加入物件偵測功能，偵測圖片和即時網路攝影機影片中的狗。

課程內容

- 如何使用 [MediaPipe Tasks](#) 在網頁應用程式中加入物件偵測工作。

建構項目

- 可偵測狗隻的網頁應用程式。您也可以使用 [MediaPipe Model Maker](#) 自訂模型，偵測所選的物件類別。

軟硬體需求

- [CodePen](#) 帳戶
 - 具備網路瀏覽器的裝置
 - 具備 JavaScript、CSS 和 HTML 基礎知識
-

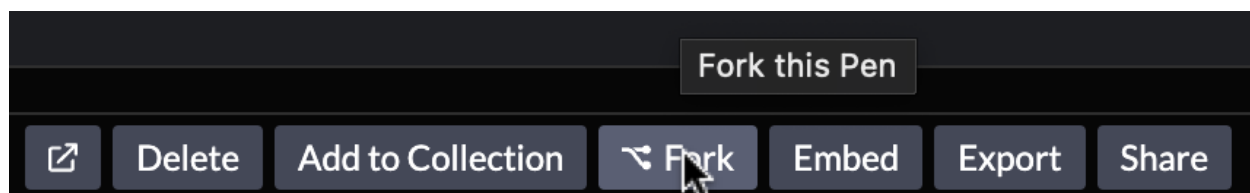
步驟 2 · 約 3 分鐘

2. 做好準備

本程式碼研究室會在 [CodePen](#) 中執行程式碼。CodePen 是一個社群開發環境，可讓您在瀏覽器中編寫程式碼，並在建構過程中檢查結果。

如要開始設定，請按照下列步驟操作：

1. 在 CodePen 帳戶中，前往這個 [CodePen](#)。您可以使用這段程式碼做為基礎，建立自己的物件偵測器。
2. 在導覽選單的 CodePen 底部，按一下 **Fork**，複製範例程式碼。



3. 在「JS」分頁中，按一下



展開箭頭，然後選取「Maximize JavaScript editor」(將 JavaScript 編輯器最大化)。在本程式碼研究室中，您只會在 JS 分頁中編輯工作，因此不需要查看 **HTML** 或 **CSS** 分頁。

查看範例應用程式

1. 在預覽窗格中，你會看到兩張狗狗的圖片，以及執行網路攝影機的選項。本教學課程使用的模型是以這兩張圖片中的三隻狗訓練而成。

Custom dog detection using the MediaPipe Object Detector task

This demo uses a custom object detection model trained using [MediaPipe Model Maker](#). You can create your own custom object detection model by following [this Colab notebook](#).

Demo: Detecting Doggies

Click on an image below to detect my dogs in the image.



Demo: Dog lookalike contest using webcam continuous detection

Do you have a dog that looks like mine? Place your dog in front of your webcam to get a real-time doogie detection!

注意：如果您想在本教學課程中建立具有更一般用途物件偵測器的網頁應用程式，請參閱「[模型](#)」。

2. 在 **JS 分頁** 中，請注意程式碼中有多個註解。舉例來說，您可以在第 15 行找到下列註解：



```
// Import the required package.
```

這些註解會指出需要插入程式碼片段的位置。

3. 匯入 MediaPipe tasks-vision 套件，並新增必要變數

1. 在「JS」JS分頁中，匯入 MediaPipe `tasks-vision` 套件：

```
// Import the required package.
import { ObjectDetector, FilesetResolver, Detection } from
"https://cdn.skypack.dev/@mediapipe/tasks-vision@latest";
```

這段程式碼使用 Skypack 內容傳遞網路 (CDN) 匯入套件。如要進一步瞭解如何在 CodePen 中使用 Skypack，請參閱「[Skypack + CodePen](#)」。

在專案中，您可以使用 Node.js 和 npm，或是選擇的套件管理工具或 CDN。如要進一步瞭解需要安裝的必要套件，請參閱「[JavaScript 套件](#)」。

2. 為物件偵測器和執行模式宣告變數：

```
// Create required variables.
let objectDetector = null;
let runningMode = "IMAGE";
```

`runningMode` 變數是字串，當您偵測圖片中的物件時，該變數會設為 `"IMAGE"` 值；偵測影片中的物件時，則會設為 `"VIDEO"` 值。

4. 初始化物件偵測器

- 如要初始化物件偵測器，請在 **JS** 分頁中，於相關註解後方新增下列程式碼：

```
// Initialize the object detector.
async function initializeObjectDetector() {
  const visionFilesetResolver = await FilesetResolver.forVisionTasks(
    "https://cdn.jsdelivr.net/npm/@mediapipe/tasks-vision@latest/wasm"
  );
  objectDetector = await ObjectDetector.createFromOptions(visionFilesetResolver, {
    baseOptions: {
      modelAssetPath: "https://storage.googleapis.com/mediapipe-assets/dogs.tflite"
    },
    scoreThreshold: 0.3,
    runningMode: runningMode
  });
}
initializeObjectDetector();
```

`FilesetResolver.forVisionTasks()` 方法會指定工作適用的 [WebAssembly \(Wasm\)](#) 二進位檔位置。

`ObjectDetector.createFromOptions()` 方法會例項化物件偵測器。您必須提供用於偵測的模型路徑。在本例中，狗隻偵測模型託管於 [Cloud Storage](#)。

注意：如要使用更一般用途的物件偵測模型，請參閱「[模型](#)」。

`scoreThreshold` 屬性已設為 `0.3` 值。也就是說，模型會針對信賴水準為 30% 以上的任何偵測到的物件傳回結果。您可以視應用程式需求調整這個門檻。

`runningMode` 屬性會在 `ObjectDetector` 物件初始化時設定。日後可以視需要變更這項設定和其他選項。

5. 對圖片執行預測

- 如要在圖片上執行預測，請前往 `handleClick()` 函式，然後在函式主體中新增下列程式碼：

```
// Verify object detector is initialized and choose the correct running mode.
if (!objectDetector) {
    alert("Object Detector still loading. Please try again");
    return;
}

if (runningMode === "VIDEO") {
    runningMode = "IMAGE";
    await objectDetector.setOptions({ runningMode: runningMode });
}
```

這段程式碼會判斷物件偵測器是否已初始化，並確保圖片已設定執行模式。

偵測物件

- 如要在圖片中偵測物件，請在 `handleClick()` 函式主體中加入下列程式碼：

```
// Run object detection.
const detections = objectDetector.detect(event.target);
```

注意： `objectDetector.detect()` 方法會封鎖主執行緒，因此在主執行緒上執行時會封鎖 UI。

以下程式碼片段包含這項工作的輸出資料範例：

```
ObjectDetectionResult:
Detection #0:
Box: (x: 355, y: 133, w: 190, h: 206)
Categories:
  index      : 17
  score      : 0.73828
  class name : aci
Detection #1:
Box: (x: 103, y: 15, w: 138, h: 369)
Categories:
  index      : 17
  score      : 0.73047
  class name : tikka
```

處理及顯示預測結果

- 在 `handleClick()` 函式主體的結尾，呼叫 `displayImageDetections()` 函式：


```
// Call the displayImageDetections() function.
displayImageDetections(detections, event.target);
```

2. 在 `displayImageDetections()` 函式的主體中，新增下列程式碼來顯示物件偵測結果：

```
// Display object detection results.

const ratio = resultElement.height / resultElement.naturalHeight;

for (const detection of result.detections) {
  // Description text
  const p = document.createElement("p");
  p.setAttribute("class", "info");
  p.innerText =
    detection.categories[0].categoryName +
    " - with " +
    Math.round(parseFloat(detection.categories[0].score) * 100) +
    "% confidence.";
  // Positioned at the top-left of the bounding box.
  // Height is that of the text.
  // Width subtracts text padding in CSS so that it fits perfectly.
  p.style =
    "left: " +
    detection.boundingBox.originX * ratio +
    "px;" +
    "top: " +
    detection.boundingBox.originY * ratio +
    "px;" +
    "width: " +
    (detection.boundingBox.width * ratio - 10) +
    "px;";
  const highlighter = document.createElement("div");
  highlighter.setAttribute("class", "highlighter");
  highlighter.style =
    "left: " +
    detection.boundingBox.originX * ratio +
    "px;" +
    "top: " +
    detection.boundingBox.originY * ratio +
    "px;" +
    "width: " +
    detection.boundingBox.width * ratio +
    "px;" +
    "height: " +
    detection.boundingBox.height * ratio +
    "px;";

  resultElement.parentNode.appendChild(highlighter);
  resultElement.parentNode.appendChild(p);
}
```

這項函式會在圖片中偵測到的物件上顯示定界框。這項功能會移除先前的所有醒目顯示效果，然後建立並顯示 `<p>` 標記，醒目顯示偵測到的每個物件。

測試應用程式

在 CodePen 中變更程式碼時，預覽窗格會在儲存後自動重新整理。如果已啟用自動儲存功能，應用程式可能已重新整理，但建議再次重新整理。

如要測試應用程式，請按照下列步驟操作：

1. 在預覽窗格中，按一下每張圖片即可查看預測結果。定界框會顯示狗狗名稱以及模型的信賴度。
2. 如果沒有顯示定界框，請開啟 Chrome 開發人員工具，然後檢查「Console」面板是否有錯誤，或回頭查看先前的步驟，確認沒有遺漏任何操作。

Custom dog detection using the MediaPipe Object Detector task

This demo uses a custom object detection model trained using [MediaPipe Model Maker](#). It is designed to accompany [this Codelab](#). You can create your own custom object detection model by following [this Colab notebook](#).

Demo: Detecting Doggies

Click on an image below to detect my dogs in the image.



6. 對網路攝影機即時影像執行預測

偵測物件

- 如要在即時網路攝影機影片中偵測物件，請前往 `predictWebcam()` 函式，然後在函式主體中加入下列程式碼：

```
// Run video object detection.
// If image mode is initialized, create a classifier with video runningMode.
if (runningMode === "IMAGE") {
  runningMode = "VIDEO";
  await objectDetector.setOptions({ runningMode: runningMode });
}
let nowInMs = performance.now();

// Detect objects with the detectForVideo() method.
const result = await objectDetector.detectForVideo(video, nowInMs);

displayVideoDetections(result.detections);
```

無論您是在串流資料或完整影片上執行推論，影片物件偵測都會使用相同方法。`detectForVideo()` 方法與用於相片的 `detect()` 方法類似，但包含與目前影格相關聯的時間戳記額外參數。這項函式會即時執行偵測，因此您要將目前時間當做時間戳記傳遞。

處理及顯示預測結果

- 如要處理及顯示偵測結果，請前往 `displayVideoDetections()` 函式，然後在函式主體中新增下列程式碼：

```

// Display video object detection results.
for (let child of children) {
  liveView.removeChild(child);
}
children.splice(0);

// Iterate through predictions and draw them to the live view.
for (const detection of result.detections) {
  const p = document.createElement("p");
  p.innerText =
    detection.categories[0].categoryName +
    " - with " +
    Math.round(parseFloat(detection.categories[0].score) * 100) +
    "% confidence.";
  p.style =
    "left: " +
    (video.offsetWidth -
      detection.boundingBox.width -
      detection.boundingBox.originX) +
    "px;" +
    "top: " +
    detection.boundingBox.originY +
    "px;" +
    "width: " +
    (detection.boundingBox.width - 10) +
    "px;";

  const highlighter = document.createElement("div");
  highlighter.setAttribute("class", "highlighter");
  highlighter.style =
    "left: " +
    (video.offsetWidth -
      detection.boundingBox.width -
      detection.boundingBox.originX) +
    "px;" +
    "top: " +
    detection.boundingBox.originY +
    "px;" +
    "width: " +
    (detection.boundingBox.width - 10) +
    "px;" +
    "height: " +
    detection.boundingBox.height +
    "px;";

  liveView.appendChild(highlighter);
  liveView.appendChild(p);

  // Store drawn objects in memory so that they're queued to delete at next call.
  children.push(highlighter);
  children.push(p);
}

```

```
}  
}
```

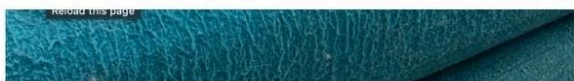
這段程式碼會移除先前的醒目顯示效果，然後建立並顯示 `<p>` 標記，醒目顯示偵測到的每個物件。

測試應用程式

如要測試即時物件偵測功能，最好使用模型訓練時用過的狗狗圖片。

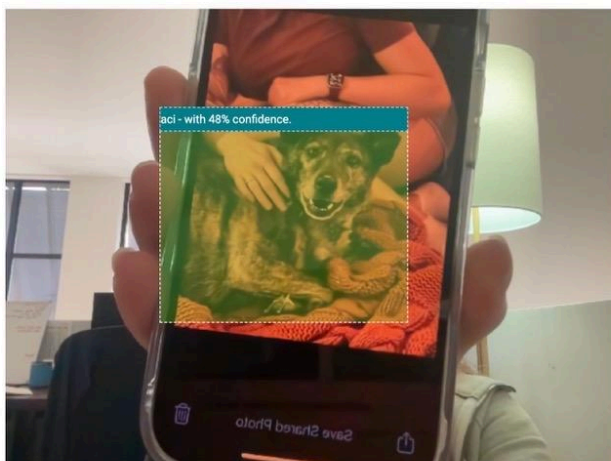
如要測試應用程式，請按照下列步驟操作：

1. 將其中一張狗狗相片下載到手機。
2. 在預覽窗格中，按一下「啟用網路攝影機」。
3. 如果瀏覽器顯示對話方塊，要求您授予網路攝影機存取權，請授予權限。
4. 將手機上狗狗的圖片放在網路攝影機前方。定界框會顯示狗狗的名稱和模型的可信度。
5. 如果沒有顯示定界框，請開啟 Chrome 開發人員工具，然後檢查「Console」面板是否有錯誤，或回頭查看先前的步驟，確認沒有遺漏任何操作。



Demo: Dog lookalike contest using webcam continuous detection

Do you have a dog that looks like mine? Place your dog in front of your webcam to get a real-time doggie detection! When ready click "enable webcam" below and accept access to the webcam.



步驟 7 · 約 1 分鐘

7. 恭喜

恭喜！您已建構可偵測圖片中物件的網頁應用程式。詳情請參閱 [CodePen 上的應用程式完整版本](#)。

瞭解詳情

- [網頁物件偵測](#)
 - [MediaPipe Studio](#)
 - [MediaPipe Model Maker](#)
 - [使用 MediaPipe Model Maker 訓練自訂物件偵測模型](#)
 - [使用 TensorFlow.js 在瀏覽器中進行自訂物件偵測](#)
-